

MetricaOne

Documentação Técnica

Sistema de Gestão de Indicadores por Loja

Versão do documento: 1.0

Público-alvo: desenvolvedores e equipe técnica
2026

Documentação Técnica

VISÃO GERAL

1. O que é o MetricaOne

O MetricaOne é uma aplicação web para acompanhamento de indicadores de desempenho de lojas. Cada loja recebe pontuações em diversas áreas operacionais (Vendas, Quebras, Prevenção, RH, entre outras), e o sistema consolida esses dados em rankings, comparativos e gráficos atualizados automaticamente.

O módulo central é o **Painel de Ouro**, responsável pelos rankings (anual, semestral, de superação) e pelo dashboard comparativo entre anos.

2. Arquitetura

A aplicação segue uma arquitetura simples e direta, sem framework pesado de front-end:

- Front-end:** HTML, CSS e JavaScript puro (vanilla). Sem build step obrigatório.
- Back-end e banco:** Supabase (PostgreSQL gerenciado na nuvem), acessado diretamente pelo cliente JavaScript.
- Autenticação:** Supabase Auth, com login por matrícula ou e-mail.
- Gráficos:** Chart.js (carregado via CDN).
- Ícones:** Font Awesome 6.
- Operações administrativas sensíveis:** isoladas em uma Edge Function (Deno) no Supabase.

Princípio de design: os dados são guardados sempre com `ano` e `mês` separados. Isso permite qualquer recorte temporal (mês, semestre, ano) apenas filtrando os registros, sem duplicação de estrutura.

3. Estrutura de pastas

```
/
├── index.html           Ponto de entrada da aplicação
├── login.html          Tela de login
├── app/
│   ├── app.js          Inicialização, Supabase, sessão
│   ├── login.js        Lógica de login (matrícula/e-mail)
│   ├── logger.js       Logs com mascaramento de dados (LGPD)
│   └── painel-ouro/
│       ├── painel-ouro.js  Núcleo: abas, rankings, dashboard
│       ├── sidebar-painel-ouro.js  Menu lateral
│       ├── calculosvendas.js  Motor de cálculo de pontos
│       ├── trava-edicao.js    Trava de edição por semestre
│       └── lancamento/      Módulos de lançamento por área
├── css/                Folhas de estilo
├── supabase/           Scripts SQL (RLS, importação, retenção)
└── edge-functions/     Funções administrativas (Deno)
```

4. Principais tabelas do banco

Tabela	Função
<code>painel_ouro_lojas</code>	Cadastro das lojas (código, nome, regional, ativo).
<code>painel_ouro_areas</code>	Áreas avaliadas (slug, nome, peso máximo, ordem).
<code>painel_ouro_indicadores_config</code>	Configuração de indicadores e seus pontos.
<code>painel_ouro_resultados</code>	Resultados lançados por loja/área/ano/mês, com detalhamento em campo JSON.
<code>usuarios</code>	Usuários do sistema, com perfil e vínculo de autenticação.
<code>auditoria</code>	Registro de eventos sensíveis (trilha de auditoria).

A tabela central é a `painel_ouro_resultados`. Cada registro guarda a pontuação obtida e a máxima, além de um campo `sub_resultados` (JSON) com o detalhamento por indicador. Há uma restrição de unicidade por `loja_codigo + area_slug + ano +`

mes

5. Como rodar localmente

1. Clone o repositório do GitHub.
2. Não há instalação obrigatória de dependências para o front-end — é HTML/JS puro.
3. Sirva a pasta com um servidor estático simples. Exemplos:

```
# com Python
python3 -m http.server 5501

# ou com a extensão Live Server (VS Code)
```

4. Abra `http://localhost:5501` no navegador.
5. As credenciais públicas do Supabase já estão configuradas em `app/app.js` (chave publicável, segura para o front-end).

Atenção à segurança: apenas a chave *publicável* deve ficar no front-end. A chave secreta (service role) nunca deve aparecer no código do cliente — ela fica somente como segredo da Edge Function no Supabase.

6. Cálculo de pontuação

O motor de cálculo fica em `calculosvendas.js` (objeto global `window.poCalculos`). Ele cuida de: interpretar valores em formato brasileiro (R\$, vírgula decimal), verificar se uma meta foi atingida, formatar moeda e percentual. Cada área de lançamento aplica suas próprias regras de pontos, e o total do mês é a soma das áreas.

Ordem de carga importa: em `index.html`, o arquivo `calculosvendas.js` deve ser carregado antes dos módulos de lançamento, pois eles dependem dele.

7. Trava de edição por período

Implementada em `trava-edicao.js`. Quando um semestre encerra, seus lançamentos ficam protegidos. Para editar, é exigida a confirmação de um usuário com perfil **master**, validada em um cliente Supabase temporário (sem derrubar a sessão atual). O desbloqueio é temporário e registrado na auditoria.

8. Importação de dados históricos

Scripts na pasta `supabase/` permitem importar dados em massa. O gerador `gerar-import-semestre.js` produz um SQL que cadastra a área consolidada, remove dados antigos do período e insere o histórico — tudo em uma transação, com conferência antes do commit. O casamento de código de loja é resiliente a formatos com ou sem zero à esquerda.

9. Segurança e LGPD

- **RLS (Row Level Security):** políticas segmentadas por perfil no script `rls_segmentada.sql`.
- **Minimização de dados:** o cadastro coleta apenas o necessário; login por matrícula.
- **Mascaramento de logs:** dados pessoais são mascarados em produção (`logger.js`).
- **Auditoria:** eventos sensíveis ficam registrados e são imutáveis.
- **Retenção:** rotina para expurgar registros antigos de auditoria (`retencao_dados.sql`).

10. Manutenção e evolução

Pontos de atenção para quem for evoluir o sistema:

- Cache-busting: os scripts em `index.html` usam um parâmetro de versão (`?v=...`); atualize-o ao publicar mudanças para forçar o recarregamento.
- A separação por ano é automática: ao virar o ano, o painel passa a operar no novo ano, e os anteriores permanecem no histórico, acessíveis pelo seletor de ano.
- Os estilos do Painel de Ouro são injetados via JavaScript dentro de `painel-ouro.js`.